

Advanced Data Structures and Algorithm Analysis

2026.01.01

Warning

- If not mentioned, we assume arrays are 1-indexed.
- Some solutions are out-of-date, I am disappointed at this courseware.
- I omit some contents about data structures (because it's trivial for me) and algorithms (dp, greedy, etc).

Summary

- NP-Completeness
- Approximation Algorithms
- Local Search
- Randomized Algorithms
- Parallel Algorithms
- External Sorting

1 Parallel Algorithms

Abstract

- “大功率跑车”

——王灿老师

- Balance the W, T and P .

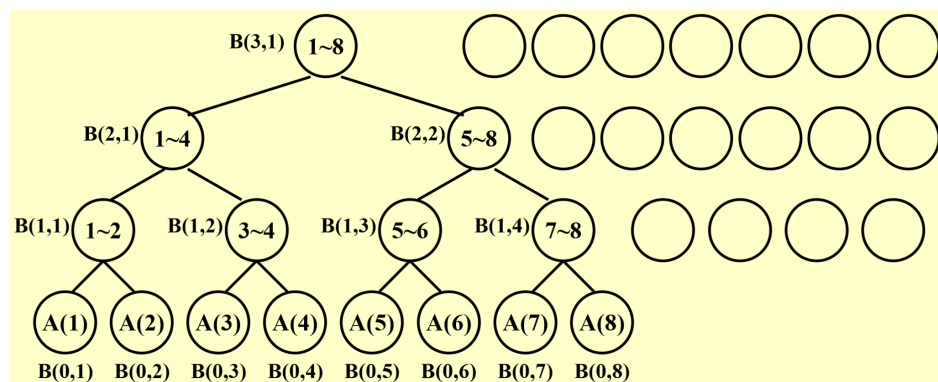
Info

- Work load $W(n)$: total number of operations.
- Time complexity $T(n)$: worst-case running time.
- Processors $P(n)$: total number of processors.

1.1 Summation

Question

Given an array A of size n , compute the summation of all elements in A .



$$B(h, i) = B(h-1, 2i-1) + B(h-1, 2i) \quad (1)$$

```

for  $P_i, 1 \leq i \leq n$  pardo
   $B(0, i) := A(i)$ 
for  $h = 1$  to  $\log n$ 
  for  $P_i, 1 \leq i \leq n/2^h$  pardo
     $B(h, i) := B(h-1, 2i-1) + B(h-1, 2i)$ 
for  $i = 1$  pardo
  output  $B(\log n, 1)$ 

```

$T(n) = O(\log n)$, $W(n) = O(n)$ and $P(n) = O(n)$.

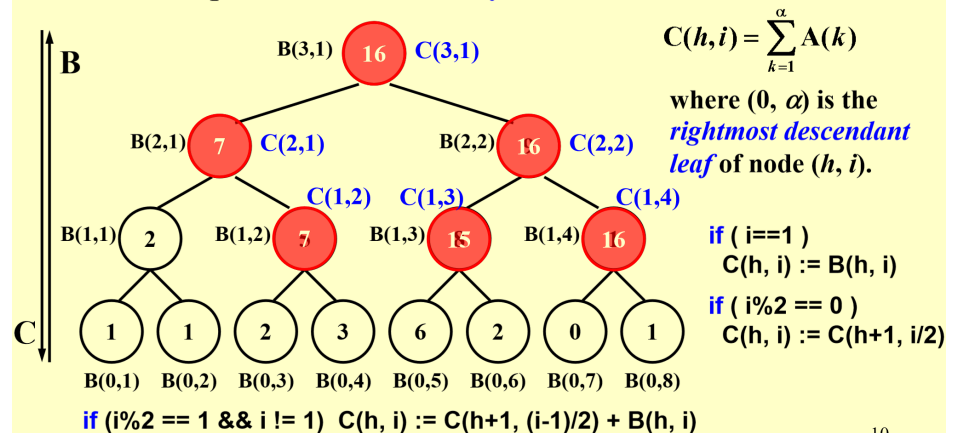
1.2 Prefix Sums

Question

Given an array A of size n , compute the prefix sums array S where $S_i = \sum_{j=1}^i A_j$ for $1 \leq i \leq n$.

Based on **Summation** problem, we design a two-phase algorithm:

Technique: Balanced Binary Trees



$$C(h, i) = \begin{cases} B(h, i) & \text{if } i = 1 \\ C(h+1, \frac{i-1}{2}) + B(h, i) & \text{if } i > 1 \wedge i \text{ is odd} \\ C(h+1, \frac{i}{2}) & \text{if } i \text{ is even} \end{cases} \quad (2)$$

$T(n) = O(\log n)$, $W(n) = O(n)$ and $P(n) = O(n)$.

1.3 Merging

Question

Given two sorted arrays A and B of size n , merge them into a sorted array C of size $2n$.

First, we transform the problem into the **Ranking** problem.

Merging $\Rightarrow$ Ranking

$\text{RANK}(j, A) = i$, if $A(i) < B(j) < A(i+1)$, for $1 \leq i < n$
 $\text{RANK}(j, A) = 0$, if $B(j) < A(1)$
 $\text{RANK}(j, A) = n$, if $B(j) > A(n)$

The **ranking problem**, denoted $\text{RANK}(A, B)$ is to compute:

1. $\text{RANK}(i, B)$ for every $1 \leq i \leq n$, and
2. $\text{RANK}(i, A)$ for every $1 \leq i \leq n$

Claim: Given a solution to the ranking problem, the merging problem can be solved in $O(1)$ time and $O(n+m)$ work.

```

for  $P_i, 1 \leq i \leq n$  pardo
   $C(i + \text{RANK}(i, B)) := A(i)$ 
for  $P_i, 1 \leq i \leq n$  pardo
   $C(i + \text{RANK}(i, A)) := B(i)$ 

```

Important

Binary Search

```

for  $P_i, 1 \leq i \leq n$  pardo
   $\text{RANK}(i, B) := \text{BS}(A(i), B)$ 
   $\text{RANK}(i, A) := \text{BS}(B(i), A)$ 

```

$T(n) = O(\log n)$
 $W(n) = O(n \log n)$

Serial Ranking

```

i = j = 0;
while (  $i \leq n \parallel j \leq m$  ) {
  if (  $A(i+1) < B(j+1)$  )
     $\text{RANK}(++i, B) = j$ ;
  else  $\text{RANK}(++j, A) = i$ ;
}

```

$T(n) = W(n) = O(n+m)$

- Binary Search: $T(n) = O(\log n)$, $W(n) = O(n \log n)$ and $P(n) = O(n)$.
- Serial Ranking: $T(n) = O(n)$, $W(n) = O(n)$ and $P(n) = O(1)$.

1.4 Parallel Ranking

Question

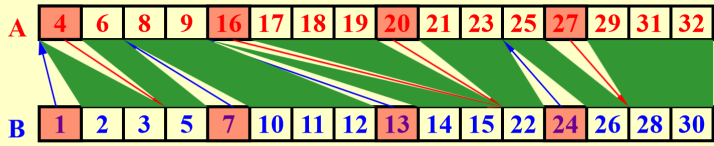
Given two sorted arrays A and B of size n , the ranking problem is to find for each element A_i , the number of elements in B that are less than A_i .

👉 Parallel Ranking

Assume that $n = m$; and that $A(n+1)$ and $B(n+1)$ are each larger than both $A(n)$ and $B(n)$.

Stage 1: Partitioning $p = n / \log n$

- $A_Select(i) = A(1 + (i-1)\log n)$ for $1 \leq i \leq p$ $T = O(\log n)$
- $B_Select(i) = B(1 + (i-1)\log n)$ for $1 \leq i \leq p$ $W = O(p \log n) = O(n)$
- Compute RANK for each *selected* element



Stage 2: Actual Ranking

At most $2p$ smaller sized ($O(\log n)$) problems.

$$T = O(\log n)$$

$$W = O(p \log n) = O(n)$$

$$T(n) = O(\log n), W(n) = O(n) \text{ and } P(n) = O\left(\frac{n}{\log n}\right).$$

1.5 Maximum Finding

- Question -

Given an array A of size n , find the maximum element in A .

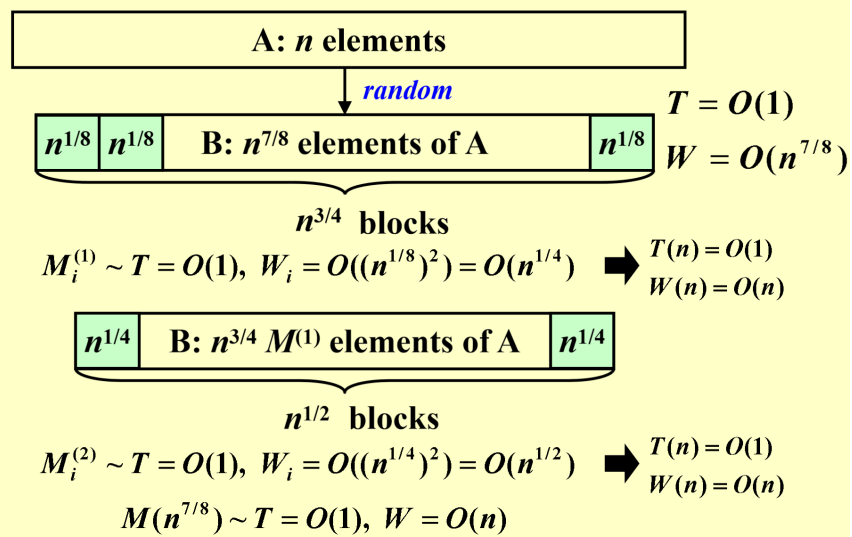
- Solution 1: Replace + by max in the **Summation** problem.
 - ▶ $T(n) = O(\log n)$, $W(n) = O(n)$.
- Solution 2: Compare all pairs.
 - ▶ $T(n) = O(1)$, $W(n) = O(n^2)$ and $P(n) = O(n^2)$.
 - ▶ Solve access conflicts (Processors try to write to the same location):
 - Test and Set , Fetch and Add , Compare and Swap , etc.
- Solution 3: Divide into $\sqrt{n}$ sub-problems, and use **Solution 2**.
 - ▶ $T(n) = T(\sqrt{n}) + O(1) \Rightarrow T(n) = O(\log \log n)$.
 - ▶ $W(n) = \sqrt{n}W(\sqrt{n}) + O(n) \Rightarrow W(n) = O(n \log \log n)$.
 - ▶ $P(n) = O(n)$.
- Solution 4: We first partition A by $h = \log \log n$, using brute force to get the maximum elements in every h elements, then apply the **Solution 3**.
 - ▶ $T(n) = O\left(h + \log \log\left(\frac{n}{h}\right)\right) = O(\log \log n)$.
 - ▶ $W(n) = O\left(n + \left(\frac{n}{h}\right) \log \log\left(\frac{n}{h}\right)\right) = O(n)$.
 - ▶ $P(n) = O\left(\frac{n}{\log \log n}\right)$.
- Solution 5: Random Sampling.

- Tip -

- ▶ Randomly select $n^{\frac{7}{8}}$ elements from A , run the algo in the picture.
- ▶ Denote the maximum of the selected elements as M , delete all elements smaller than M in A .
- ▶ Repeat the procedure, until the size of A becomes 1.

👉 **Random Sampling** $T(n) = O(1)$, $W(n) = O(n)$

with *very high probability*, on an Arbitrary CRCW PRAM



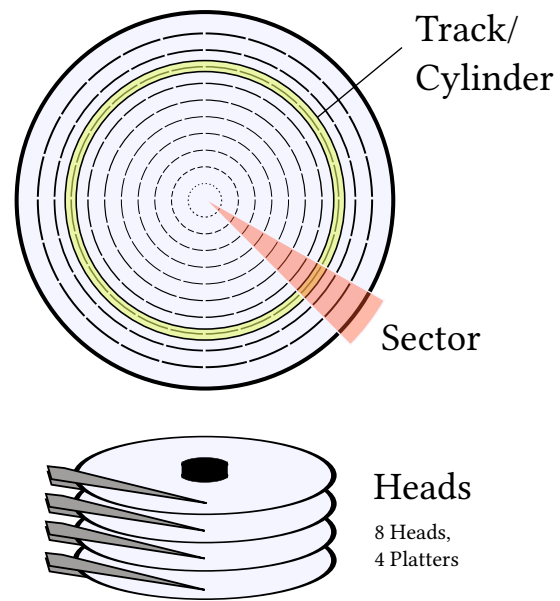
It's obviously that the expected size of A after an iteration is reduced to $n^{\frac{1}{8}}$, thus the expected number of iterations is $O(1)$ (Usually equals to 2).

$$T(n) = O(1), W(n) = O(n) \text{ and } P(n) = O(n).$$

2 External Sorting

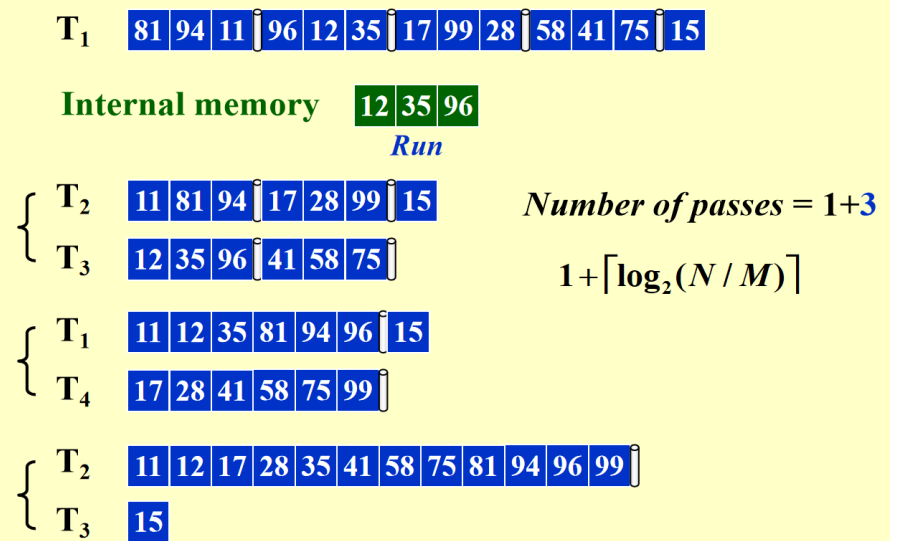
- Info -

- 硬盘 (hard disk), 磁道 (track), 扇区 (sector).



- To get a_i on hard disk, we need to find the track, then find the sector, find a_i and transmit.
- We use **tapes** (sequential access devices) and **merge** to implement external sort.

[[Example]] Suppose that the internal memory can handle $M = 3$ records at a time.



- Suppose the internal memory can handle M records at a time.
- We define a **run** (顺串) as a sorted sequence .
 - eg. In the first pass (Internal Sorting Stage), (11, 81, 94) is a run.
- **pass** (读写次数 / 趟数) means the number of loops of merging.
 - In this example, we need $1 + 3$ passes to sort the data.
 - In general, if we have N records to sort, we need $1 + \lceil \log_2(\frac{N}{M}) \rceil$ passes.

- Analysis -

- Q: How to reduce the number of passes?

Use a k -way merge (instead of 2-way merge).

- ▶ Pros: need $1 + \lceil \log_k(\frac{N}{M}) \rceil$ passes.
- ▶ Cons: need $2k$ tapes.

- Q: Can we use $k + 1$ tapes for a k -way merge?

Yes. eg. 2-way merge, we can use 3 tapes (**Fibonacci numbers**).

Claim: If the number of runs is a Fibonacci number F_N , then the best way to distribute them is to split them into F_{N-1} and F_{N-2} .

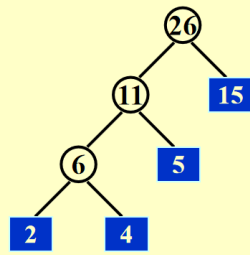
Claim: For a k -way merge, $F_N^{(k)} = F_{N-1}^{(k)} + \dots + F_{N-k}^{(k)}$
 where $F_N^{(k)} = 0$ ($0 \leq N \leq k-2$), $F_{k-1}^{(k)} = 1$

- Q: How to minimize the merge time?

[[Example]] Suppose we have 4 runs of length 2, 4, 5, and 15, respectively. How can we arrange the merging to obtain **minimum merge times**?



Huffman Tree!



$$2 \times 3 + 4 \times 3 + 5 \times 2 + 15 \times 1 = 43$$

Total merge time = $O(\text{the weighted external path length})$

- Replacement Selection

Analysis

我们考虑一个更好地构建顺串的办法——替换选择 (Replacement selection)

1. 将输入磁盘前 M 个数据输入内存 (内存中依旧使用优先队列排序)
2. 执行 DeleteMin 操作将最小值 x 输入输出磁盘中
3. 从输入磁盘再读取一个数据 y , 如果 $y \geq x$, 则 y 放入内存并上溢, 重复步骤 2; 如果 $y < x$, 则将 y 放入优先队列的死区 (dead space) 并认为优先队列大小减 1, y 参加下一个顺串的构建
4. 如果优先队列减小为 0 (End of Run) 或者是输入磁盘已经为空 (end of tape) 时, 将死区中的所有元素排序后作为一个顺串输出 (Rebuild Heap); 重复步骤 1

不难推断, 这样构建的顺串长度至少为 M , 从而达到了减少顺串数量的目的

- generate runs of average length $2M$.

- Parallel

Analysis

- In general, for a k -way merge we need $2k$ input buffers and 2 output buffers for **parallel operations**.
- For each stream, two buffers (A and B) are used. Buffer A is filled with data read from disk, while buffer B is sorted at the same time. When buffer B finishes sorting, it swaps with the now-full buffer A. The empty buffer can then be reused for reading, enabling overlap of I/O and computation.

3 Problem Set (Strange? Trash? No Human?)

3.1 Data Structures

Question

By definition, for a **light node** p in a skew heap, the number of descendants of p 's right subtree is no more than $\frac{1}{2}$ of the number of descendants of p .

F . If equals, by definition it is a **heavy node** (Consider the case the left subtree is empty and the right subtree has only one node).

Question

We have a binary counter of k bits. Each time we conduct an increment on the counter: $x \equiv x + 1 \pmod{2^k}$ and the cost of the increment is the number of bits we need to flip. For example, when $k = 3$, currently we have $x = 010$, after increment we have $x = 011$. Then this increment costs 1 because only 1 bit flips after increment. If we conduct the increment again, $x = 100$. Then this increment costs 3 because we flip 3 bits. Now we conduct n consecutive increments and estimate the total cost. Which of the following statements are TRUE?

1. If the initial value of the counter is 0, the total cost is $O(n)$.
2. If the initial value of the counter is 0, the total cost is $O(n \log k)$.
3. If $n = \Omega(k)$, the total cost is $O(n)$.
4. If $n = \Omega(k)$, the total cost is $O(n \log k)$.

- A. 2 and 4
- B. 2 and 3
- C. 1 and 3
- D. 1 and 4

C . 1 is obvious. Consider 3, for the bit i , there are at most $\lfloor \frac{n}{2^i} \rfloor$ flips. Thus the total cost is $O(k) + O(\sum_i \lfloor \frac{n}{2^i} \rfloor)$. Since $n = \Omega(k) \Rightarrow k = O(n)$, it is $O(n)$.

Question

If a leftist heap can be implemented recursively, so can its counterpart skew heap.

F . idk. Why not true?

Question

A binary tree that is not full cannot correspond to an optimal prefix code. (T/F)

T . There are 3 concepts.

- **Full Binary Tree**: every node has either 0 or 2 children.
- **Complete Binary Tree**: all levels are fully filled except possibly the last, which is filled from left to right.
- **Perfect Binary Tree**: all internal nodes have two children and all leaves are at the same level.

Thus an optimal prefix code must be a full binary tree, otherwise we can **delete the node with only one child** to reduce the cost.

3.2 Inverted File Index

Question

In a search engine, **thresholding for query** retrieves the top k documents according to their weights. (T/F)

F . It confuses two different optimizations.

- **Document Thresholding**: Filters and retrieves the top k documents based on their weights.
- **Query Thresholding**: Optimizes performance by selecting only **high-frequency or important query terms** to process, rather than the entire query.

Thus the proposition should be **thresholding for documents**.

Question

When evaluating the performance of **data retrieval**, it is important to measure the relevancy of the answer set. (T/F)

F . It confuses two different concepts.

- **Data retrieval** (eg. SQL Query): **matches a query exactly**.
 - It is **binary** (Y/N). We measure **efficiency** (how fast the system responds).
- **Information retrieval** (eg. Google Search): **'about' a certain topic**.
 - It is subjective and based on **relevancy**. We measure **effectiveness** (how relevant the answer set is to the user's needs).

3.3 Backtracking

Question

What makes the time complexity analysis of a backtracking algorithm very difficult is that **the sizes of solution spaces may vary**. (T/F)

F . Strange problem.

Question

What makes the time complexity analysis of a backtracking algorithm very difficult is that **the number of solutions that do satisfy the restriction is hard to estimate**. (T/F)

T .

3.4 Divide and Conquer

Question

Is this asymptotic upper bound for the following recursive $T(n)$ is correct?

- $T(n) = T(n^{\frac{1}{3}}) + T(n^{\frac{2}{3}}) + \log n$. Then $T(n) = O(\log n \log \log n)$.

T . By recursion tree method, the height of the tree is $O(\log \log n)$, and the cost at each level is $O(\log n)$. Thus the total cost is $O(\log n \log \log n)$.

3.5 NP-Completeness

Question

The following problem is in co-NP. (T/F)

- Given a positive integer K , is K a prime number?

T . This is P problem, and $P \subseteq \text{co-NP}$.

3.6 Approximation Algorithms

Question

In the bin packing problem, we are asked to pack a list of items L to the minimum number of bins of capacity 1. For the instance L , let $\text{FF}(L)$ denote the number of bins used by the algorithm First Fit. The instance L'

is derived from L by deleting one item from L . Then $\text{FF}(L')$ is at most of $\text{FF}(L)$. (T/F)

F . Consider the instance $L = \{0.55, 0.7, 0.55, 0.1, 0.45, 0.15, 0.3, 0.2\}$, and $L' = L - \{0.1\}$. $\text{FF}(L') = 4$, while $\text{FF}(L) = 3$.

Question

Consider the bin packing problem which uses a minimum number of bins to accommodate a given list of items. Recall that Next Fit (NF) and First Fit (FF) are two simple approaches, whose (asymptotic) approximation ratios are 2 and 1.7, respectively. Now we focus on a special class I2 of instances in which only two distinct item sizes appear. Check which of the following statements is true by applying NF and FF on I2.

A. NF and FF both have improved approximation ratios.
B. NF has an improved approximation ratio, while FF does not.
C. FF has an improved approximation ratio, while NF does not.
D. Neither of NF or FF has an improved approximation ratio.

C . For **NF** algorithm, we construct $\frac{1}{\text{eps}}$ pairs:

0.9 - eps, 2 * eps
...
0.9 - eps, 2 * eps

Obviously $\text{OPT} = \frac{1}{\text{eps}} + 1$, while $\text{NF} = \frac{2}{\text{eps}}$. Thus the approximation ratio is $\frac{2}{1+\text{eps}} \sim 2$.

For **FF** algorithm, idk. But we can feel it is improved.

3.7 Local Search

Question

For an optimization problem, given a neighborhood, if its local optimum is also a global optimum, one can reach an optimal solution with **just one step** of local improvements. (T/F)

F . The proposition means that **any local optimum is a global optimum**, thus we can always find the global optimum by **Local Search**. However, it doesn't mean we can reach the optimum in just one step.

3.8 External Sorting

Question

If only one tape drive is available to perform the external sorting, then the tape access time for any algorithm will be $\Omega(n^2)$. (T/F)

T . idk.

4 References

homework

- https://roderickshao.github.io/RoderickShao_notebook/%E8%AE%A1%E7%AE%97%E6%9C%BA%E5%9F%BA%E7%A1%80%E8%AF%BE/%E9%AB%98%E7%BA%A7%E6%95%B0%E6%8D%AE%E7%BB%93%E6%9E%84%E4%B8%8E%E7%AE%97%E6%B3%95%E5%88%86%E6%9E%90/Homework
- https://mem.ac/course/ads/correction/

notes

- https://wintermelonc.github.io/WintermelonC_Docs/zju/compulsory_courses/ADS/
- https://zhoutimemachine.github.io/note/courses/ads-final-review/
- https://birchtree2.github.io/%E5%A4%A7%E4%BA%8C%E4%B8%8B/ADS